

Competitive Programming

Section 42: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-09-14

Table of contents

1	Advanced structure diagnosis	3
2	Representation changes	4
2.1	Coordinate compression	4
2.2	Flattening a rooted tree	4
2.3	Heavy-light decomposition	4
3	Time as another dimension	5
3.1	Reverse processing	5
3.2	Persistence	5
4	Structures inside structures	6
5	Deriving a sufficient invariant	7
6	Proof template for composed solutions	8
7	Invariant-oriented testing	9
8	Complete local contest set	10
9	Model the operations first	11
10	Prove and test the invariant	12
11	Cost accounting	13
12	Exercises	14

1 Advanced structure diagnosis

These tasks are about choosing what information must remain available after many operations. The right structure follows from the query contract. At this level, one structure is often not enough: coordinates may be compressed before a Fenwick tree is built; a tree may be flattened before range updates are applied; heavy-light decomposition may turn a path into several segment-tree ranges; persistence may add a version dimension.

Describe the solution as a pipeline from original objects, through a representation change and maintained invariant, to a query decomposition.

2 Representation changes

2.1 Coordinate compression

Compression replaces each relevant value by its rank among all relevant values. It preserves equality and order, not numerical distance. It is safe for questions such as `value <= x`, but not for formulas needing actual gaps unless those gaps are stored separately. Include future update values while collecting coordinates.

2.2 Flattening a rooted tree

An Euler entry order gives every subtree one contiguous interval:

```
subtree(v) <-> [tin[v], tout[v]]
```

DFS completely finishes one child subtree before entering the next sibling. That fact proves contiguity. Subtree operations become array-range operations. A general path is not contiguous in Euler order.

2.3 Heavy-light decomposition

Choose the child with largest subtree as heavy. Heavy edges form contiguous chains. A root path crosses only $O(\log n)$ light edges: crossing a light edge upward at least doubles the subtree size. A path therefore becomes $O(\log n)$ array ranges, producing $O(\log^2 n)$ operations with an ordinary segment tree.

3 Time as another dimension

3.1 Reverse processing

Deletions are often hard while insertions are easy. If operations are known offline, start from the final state and add deleted objects back in reverse. Connectivity deletions can then become ordinary DSU unions. Track the timeline carefully: the reverse state before undoing an operation matches the forward state after that operation.

3.2 Persistence

A persistent structure retains old versions. A point update copies only one root-to-leaf path while untouched subtrees are shared:

```
old root ---- unchanged subtree
      \
       copied path ---- changed leaf
      /
new root ---- same unchanged subtree
```

A persistent segment-tree point update creates $O(\log n)$ nodes instead of copying the array. Every root represents one immutable snapshot; shared nodes must never be mutated.

4 Structures inside structures

Value-filtered and two-dimensional queries often use an outer decomposition by position and an inner sorted list or Fenwick structure by value. A merge-sort tree stores every element in $O(\log n)$ sorted node lists. A range count visits $O(\log n)$ nodes and binary-searches each list, giving $O(\log^2 n)$ query time and $O(n \log n)$ memory.

For sparse points, compress only coordinates that occur. A full coordinate grid is impossible when coordinates are large even if the number of points is small. Always calculate both the number of outer nodes visited and the cost of each inner operation.

5 Deriving a sufficient invariant

Suppose paths support addition and maximum queries. A lazy segment-tree node needs:

```
maximum = maximum actual value in this segment  
lazy     = uniform addition still owed to its descendants
```

Applying `delta` to a fully covered node adds it to both fields. Pushing sends the pending addition to the children before a partial descent. Heavy-light decomposition identifies the ranges; the lazy tree maintains them. These are separate correctness obligations.

Order statistics instead need counts. During a k-th-element descent, the left count decides whether to enter the left child or subtract that count and enter the right. The same logic works on the difference between two persistent frequency roots when that difference represents a requested subarray.

6 Proof template for composed solutions

Verify a multi-layer solution through small claims:

1. The representation maps every original query to the claimed ranges or versions.
2. Every stored node or prefix has a precise invariant.
3. Every update preserves that invariant.
4. Query combination counts each required object exactly once.
5. The number of pieces and cost per piece imply the stated complexity.

Do not replace these claims by “the data structure is standard.” Most bugs occur between two individually standard techniques.

7 Invariant-oriented testing

Feature	Adversarial shape
tree decomposition	chain, star, and balanced branches
compression	duplicates and future update values
persistence	update an old version after newer ones exist
reverse time	delete and restore a critical bridge
lazy paths	overlapping full paths and single vertices
sparse coordinates	points near both numerical extremes

Large tests must also exert the intended complexity. A chain exposes linear parent climbing and recursion depth; alternating path endpoints exercise heavy-light chain changes; many versions verify that updates copy only logarithmically many nodes.

8 Complete local contest set

Problem	Title
A	Dynamic Order Statistics
B	Range Assign and Add
C	Static Rectangle Count
D	Nested Ranges Count
E	Subtree Add, Point Query
F	Dynamic Path Maximum
G	Subtree Value Count
H	Connectivity Countdown
I	Persistent Version Sums
J	Subtree K-th Smallest
K	Sparse Rectangle Sums
L	Path Add, Path Maximum

Problems A–F establish the core coverage. Problems G–L deliberately increase in difficulty, moving from static range indexing to reversed time, persistence, sparse two-dimensional indexing, and lazy heavy-light decomposition.

9 Model the operations first

For each local problem, list every update and query, then answer:

```
Is the input static or dynamic?  
Does a query ask for an aggregate, a position, or a count of objects?  
Can coordinates be compressed without changing comparisons?  
Can queries be processed offline?  
What information must an internal node or prefix preserve?
```

Do not turn this into a lookup table from wording to structure. Derive an invariant that would let a proposed representation answer every operation. The editorial contains the local construction after you have tested yours.

10 Prove and test the invariant

State the meaning of each stored number in one sentence. Show how an update preserves it, and why the query combines stored values into exactly the desired answer. Test against a slow vector or nested-loop oracle on tiny random cases, especially repeated values, empty ranges, and coordinates near boundaries.

11 Cost accounting

Give both per-operation and total complexity. Include preprocessing, compression, sorting, and memory. A logarithmic query can still be unsuitable if a hidden rebuild happens for every operation.

12 Exercises

For A through L, write an operation contract and an invariant before coding. Also name one boundary case your brute-force oracle must include. Consult the editorial only after this attempt. Solve G–L in order: the progression is part of the exercise design.